

Absolutely. And I think the key is **not to make this another giant list of technologies**.

You already know enough Node/Express/TypeScript to start building. What you're missing is **depth + repetition across the pieces that make a backend production-grade**.

I'd make your roadmap **project-driven**, with one major project—**Eventra** is perfect—being progressively upgraded as you learn each backend concept.

Your Backend Roadmap

```
PHASE 0
Foundation & Node mastery
    ↓
PHASE 1
API engineering
    ↓
PHASE 2
PostgreSQL + data modeling
    ↓
PHASE 3
Authentication & authorization
    ↓
PHASE 4
Production error handling + validation
    ↓
PHASE 5
Testing
    ↓
PHASE 6
Caching + Redis
    ↓
PHASE 7
Events + background jobs
    ↓
PHASE 8
Emails + webhooks + payments
    ↓
PHASE 9
Security + rate limiting
    ↓
PHASE 10
Logging + observability
    ↓
PHASE 11
Docker + CI/CD
    ↓
PHASE 12
Deployment
    ↓
PHASE 13
Advanced backend / system design
```

And importantly, **we don't move on just because you've "covered" something.**

The goal is:

I can build it without constantly wondering what I'm doing.

PHASE 0 — Node.js foundation

Before everything else, I want you comfortable with Node itself.

Master

- Node runtime
- V8
- Event loop
- asynchronous programming
- promises
- `async/await`
- `Promise.all`
- modules
- environment variables
- `process`
- filesystem
- streams
- buffers
- HTTP
- npm/pnpm
- package management

Build

A tiny HTTP server **without Express**.

Then:

```
Node HTTP server
  ↓
Routing
  ↓
JSON responses
  ↓
Request parsing
  ↓
Error handling
```

This gives you a mental model of what's underneath Express.

PHASE 1 — API Engineering

Now Express/Hono becomes your tool.

You need to become extremely comfortable with:

```
Routes
Middleware
Controllers
Services
Repositories
Request
Response
Headers
Params
Query
Body
Status codes
REST
```

Build:

```
GET    /events
GET    /events/:id
POST   /events
PATCH /events/:id
DELETE /events/:id
```

Then add:

```
Pagination
Filtering
Sorting
Search
```

For example:

```
GET /events?page=2&limit=20
```

```
GET /events?category=music&city=dubai
```

```
GET /events?search=concert
```

You should understand **how and why** these work, not just copy controller code.

PHASE 2 — PostgreSQL Deep Dive

This is one of your biggest priorities.

You specifically said:

"pgsql, I need to dive deeper"

I agree.

Don't just learn PostgreSQL through Prisma.

Learn **SQL first, Prisma second**.

SQL

Master:

```
CREATE
INSERT
SELECT
UPDATE
DELETE

WHERE
ORDER BY
GROUP BY
HAVING

JOIN
LEFT JOIN
RIGHT JOIN

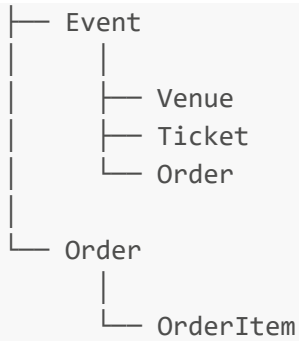
Subqueries
CTEs
Aggregations

Indexes
Constraints
Foreign keys
Unique constraints

Transactions
Isolation
Locks
```

Then database design:

```
User
|
```



Then Prisma.

Your Eventra database should eventually involve:

```
users
roles
events
venues
tickets
orders
order_items
payments
sessions
refresh_tokens
webhook_events
notifications
```

This phase should make you confident enough to look at a requirement and design the database yourself.

PHASE 3 — Authentication & Authorization

This deserves its own phase.

You mentioned OAuth specifically.

Start with:

Authentication

```
Register
Login
Logout
Password hashing
Sessions
JWT
```

Access tokens
Refresh tokens

Then:

Authorization

User
Organizer
Admin

Understand:

Authentication
↓
"Who are you?"

Authorization
↓
"What are you allowed to do?"

Then go into:

OAuth

Understand the actual flow rather than memorizing code:

Your app
↓
"Login with Google"
↓
Google
↓
User authenticates
↓
Authorization code
↓
Your backend
↓
Exchange code
↓
Access token
↓
User information
↓
Create/login local user

Then learn:

```
OAuth 2.0
OpenID Connect
Authorization Code Flow
PKCE
State parameter
Access tokens
Refresh tokens
Scopes
```

You don't need to become an OAuth protocol researcher. 🤪

You need to be able to **implement and debug it confidently**.

PHASE 4 — Validation + Error Handling

This is one of your stated weak points.

We're going to go **deep** here.

You need to understand the difference between:

```
Programming error
Validation error
Authentication error
Authorization error
Not found
Conflict
Business logic error
Database error
External service error
```

Then create your own error system.

Something conceptually like:

```
AppError
├─ ValidationError
├─ AuthenticationError
├─ AuthorizationError
├─ NotFoundError
├─ ConflictError
└─ ExternalServiceError
```


Then:

```
Request
  ↓
Controller
  ↓
Service
  ↓
Error
  ↓
Error Middleware
  ↓
HTTP Response
```

You'll learn:

```
try/catch
next(error)
async errors
custom errors
error middleware
error codes
error responses
logging errors
operational vs programming errors
```

This is one of the areas where I want you to eventually say:

"Something broke? I know exactly where that error goes."

PHASE 5 — Testing

Don't leave testing until the end.

You'll learn:

```
Unit tests
Integration tests
API tests
Database testing
Mocks
Spies
Fixtures
Test database
```

Stack:

```
Vitest
+
Supertest
+
PostgreSQL
```

Eventually:

```
POST /auth/register
  ↓
TEST
POST /auth/login
  ↓
TEST
GET /events
  ↓
TEST
POST /events
  ↓
TEST
Unauthorized request
  ↓
TEST
Invalid data
  ↓
TEST
```

The objective isn't "100% coverage."

It's:

I can change my backend without being terrified that I broke something.

That's confidence.

PHASE 6 — Redis + Caching

Now Redis.

Start with the fundamentals:

- Keys
- Values
- TTL
- Expiration
- Hashes
- Lists
- Sets
- Sorted sets
- Pub/Sub

Then use cases.

API caching

```
graph TD
    Request --> Redis{Redis?}
    Redis --> YES[YES → return cached data]
    YES --> NO[NO]
    NO --> PostgreSQL[PostgreSQL]
    PostgreSQL --> Store[Store in Redis]
    Store --> Return[Return]
```

Learn:

- Cache-aside
- TTL
- Cache invalidation
- Cache keys
- Cache stampede
- Cache consistency

Then Redis for:

- Sessions
- Rate limiting
- Queues
- Pub/Sub
- Temporary data

This is where Redis starts becoming much more than "a cache."

PHASE 7 — Events + Background Jobs

You specifically mentioned:

events, background jobs

These are extremely important.

Learn the difference between:

Synchronous

```
Request
  ↓
Create order
  ↓
Send email
  ↓
Return response
```

and:

Asynchronous

```
Request
  ↓
Create order
  ↓
Create job
  ↓
Return response

      ↓
    Worker
      ↓
  Send email
```

Then learn application events:

```
UserRegistered
OrderCreated
PaymentCompleted
```

TicketPurchased
EventPublished

And:

Event
↓
Listener
↓
Job
↓
Worker

Then:

Redis
↓
BullMQ
↓
Worker

Learn:

Retries
Backoff
Delayed jobs
Failed jobs
Dead-letter concepts
Idempotency
Concurrency

Idempotency is particularly important once we reach payments and webhooks.

PHASE 8 — Emails + Webhooks + Payments

Now we're entering serious backend territory.

Emails

Learn:

Transactional email
Templates

Queues
Retries
Email providers

Architecture:

```
graph TD
    A[Order created] --> B[Event]
    B --> C[Email job]
    C --> D[Redis queue]
    D --> E[Worker]
    E --> F[Email provider]
```

Webhooks

You specifically said you've never properly gone through these.

We will.

Understand:

What is a webhook?
Webhook endpoint
Payload
Signature
Verification
Retries
Duplicate events
Idempotency
Webhook storage
Webhook processing

For example:

```
graph TD
    A[Payment Provider] --> B[POST /webhooks/payment]
    B --> C[Verify signature]
    C --> D[Check event ID]
```

```
↓  
Store event  
↓  
Process event  
↓  
Update order  
↓  
Return 200
```

You'll eventually understand why **you cannot blindly trust a webhook request**.

PHASE 9 — Security + Rate Limiting

This covers your:

throttling, limiters

Learn the distinction between:

```
Rate limiting  
Throttling  
Quotas
```

Then implement:

```
100 requests/minute
```

or:

```
5 login attempts/minute
```

Redis becomes useful here.

Understand algorithms conceptually:

```
Fixed window  
Sliding window  
Token bucket  
Leaky bucket
```

Then security:

- CORS
- CSRF
- Helmet
- Rate limiting
- Input validation
- Password security
- Secrets
- HTTPS
- SQL injection
- XSS
- SSRF
- Brute force protection

PHASE 10 — Logging + Observability

You mentioned logging.

Don't think of logging as:

```
console.log("user created")
```



Learn structured logging.

Something conceptually like:

```
{  
  "level": "info",  
  "event": "order.created",  
  "orderId": "123",  
  "userId": "456",  
  "requestId": "abc"  
}
```

Learn:

- Log levels
- Structured logs
- Request IDs
- Correlation IDs
- Error logging
- Performance logging

Health checks
Metrics

Eventually you should be able to answer:

"What happened to order #123?"

by following the logs.

PHASE 11 — Docker

Now containerize your system.

You should be comfortable with:

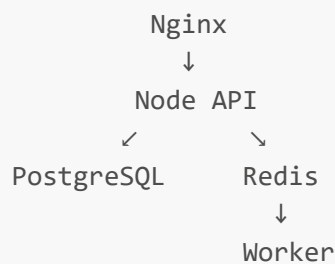
Dockerfile
Images
Containers
Volumes
Networks
Environment variables
Docker Compose

Your local Eventra environment:

Docker Compose

- |— API
- |— PostgreSQL
- |— Redis
- |— Worker

Eventually:



PHASE 12 — CI/CD

You said:

"CI/CD, need understanding"

We'll make this practical.

Understand:

```
Developer
  ↓
git push
  ↓
GitHub
  ↓
CI
  ↓
Install
  ↓
Lint
  ↓
Test
  ↓
Build
  ↓
Deploy
```

Learn:

```
GitHub Actions
Environment variables
Secrets
Build pipelines
Testing pipelines
Deployment pipelines
Rollback concepts
```

You should eventually have:

```
Pull Request
  ↓
Tests
  ↓
Lint
  ↓
Build
```

↓
Merge
↓
Deploy

PHASE 13 — Deployment

Finally:

Linux server
SSH
Nginx
Docker
Domain
HTTPS
Reverse proxy
Process management
Database
Backups
Environment configuration

Then AWS.

Don't try to learn AWS as a universe.

Start with:

EC2
RDS
S3
IAM
CloudWatch
Route 53

And then... System Design

Once you've done all that, we start asking bigger questions.

For example:

Eventra suddenly has 100,000 users. What breaks?

Then:

Load balancing
Horizontal scaling
Database replicas
Caching
Queues
Workers
CDNs
Object storage
Rate limiting
Distributed systems
Consistency
Availability
Fault tolerance

This is where your backend knowledge starts becoming **engineering knowledge**.



The project we'll use

I strongly recommend we make **Eventra your backend laboratory**.

Instead of creating 30 little tutorial projects:

```
Eventra
|
├── Authentication
├── Users
├── Roles
├── Events
├── Venues
├── Tickets
├── Orders
├── Payments
├── Uploads
├── Search
├── Pagination
├── Filtering
├── Validation
├── Authorization
├── OAuth
├── Webhooks
├── Emails
├── Events
├── Background jobs
├── Redis
├── Caching
├── Transactions
├── Rate limiting
├── Logging
```

- Testing
- Docker
- CI/CD
- Deployment

Every new concept gets **implemented into the same system**.

That's going to be much more valuable than simply watching tutorials.

The progression I want for you

Think of your confidence developing like this:

Level 1 — CRUD Builder

I can create APIs.

↓

Level 2 — Application Builder

I can build a complete backend.

↓

Level 3 — Production Builder

I can make it secure, tested,
observable and deployable.

↓

Level 4 — Backend Engineer

I understand why the system
works the way it does.

↓

Level 5 — Systems Engineer

```
I can reason about scale,  
failure, concurrency,  
performance and distributed systems.
```

Your immediate target is Level 3.

Not microservices. Not Kubernetes. Not 50 AWS services. Not Kafka just because it's popular.

Get **really damn good at building a production-quality monolithic backend first.**

And honestly, looking at the gaps you've listed, I think this is a **very achievable path for you** because you aren't starting from zero. You already have Node, TypeScript, Express, Prisma, React, databases, and several real application ideas in your toolkit.

The next step isn't *more technologies*.

It's **depth, implementation, debugging, and repetition.**